Event row contract — v1

> **Status: proposed, awaiting sign-off.** Agree this before the generator is built. Once agreed, the generator and the engine can be built in parallel rather than one waiting on the other.

The contract is enforced in code, not just here: both sides validate against `reco/domain/event_contract.py`. A field renamed on one side fails the other side's tests immediately, rather than on demo day.

One row in `ml_demo_events`

```
{
  "user_id":               "u01",
  "event_type":            "view",
  "categories":            ["fashion & apparel"],
  "timestamp":             ISODate("2026-08-29T12:11:59Z"),
  "weight":                2,
  "view_duration_seconds": 7.5,
  "order_id":              null
}
```

The five every row carries

Field	Type	Notes
`user_id`	string	Text form, not ObjectId
`event_type`	string	One of: `view`, `search`, `click`, `cart`, `purchase`, `purchase_reversal`
`categories`	**array** of string	`toCategoryKey()` keys — lowercase, trimmed, spaces preserved, no underscores. Array even when there is one
`timestamp`	date	UTC, timezone-aware
`weight`	number	Written by the generator. **Never read by the engine**

The two that appear only where they apply

Field	On	Notes
`view_duration_seconds`	`view` rows only	P-02 is applied to it by the method, not the generator
`order_id`	`purchase` and `purchase_reversal` rows	Required on both

Five on every row, seven in total. Both counts are correct and describe the same contract.

Three points that need explicit agreement

1. `categories` is plural, and an array

The task list says "category". It has to be `categories`, an array.

A post carrying several categories contributes **full weight to each**, not a divided share — that is U03's entire purpose, and it's one of the narrated demo users. A singular `category` field cannot express it. If the generator emits singular, U03 needs a second code path, and a special case built to make a demonstration work is a finding about the build, not a shortcut.

The array holds one element in the ordinary case, so this costs nothing.

2. A reversal is an `event_type`, not a flag

`purchase_reversal` is its own event type rather than a boolean on a purchase row. That keeps the contract at seven fields, makes a row self-describing when someone reads it in Atlas, and means the reversal cannot be created by accident through a defaulted flag.

Its weight is the purchase weight **negated**, and it is aged from the **original order's creation**, never from the cancellation — which is why `order_id` is mandatory on it. A cancellation is always more recent than the order it cancels, so it decays less, so it is always larger; aged from the cancellation the user ends up below someone who never ordered and the raw goes negative.

The reversal is **not** P-07 negative feedback. P-07 is hide/report/skip and is 0 at launch; multiplying the reversal by it would zero the reversal silently.

3. `weight` is written and never read

The generator writes it so the data checks can do arithmetic a reader can follow. The engine derives every weight from **P-06 keyed on** `event_type` and ignores the field — `domain/events.Event` has nowhere to put it and forbids extra fields, so an adapter cannot carry one through even by mistake.

If the engine read it, P-06 would stop being configuration: retuning a weight would change no score, the config layer would be decorative, and nothing would fail loudly to say so.

The one legitimate use is a check asserting the generator's belief matches P-06: `DemoEventRow.expected_weight(tunables)` exists for exactly that.

What must never appear in this collection

A declared interest. Signup picks are a statement of preference, not an action. They carry no timestamp, contribute a constant floor of 10 (P-25), and must never be written to the event stream — where Method A's decay term would consume them. They live on the user record's `interest` array and nowhere else.

This is the single easiest thing in the build to get wrong (G6), so the generator carries a test for it and `event_contract.is_declared_interest_row()` is there to make the assertion cheap.

The three sibling collections

The same profile switch moves all four sources together, so the generator writes all four:

Collection	Carries
`ml_demo_events`	This contract
`ml_demo_posts`	`_id`, `categories` — so the live join path can be exercised too
`ml_demo_users`	`user_id`, `interest` (array), `created_at` — the P-25 floor source
`ml_demo_orders`	`_id`, `created_at`, `order_status` — the timestamp a reversal is aged from

---